Here is a structured, step-by-step guide to building the **Student Registration Portal** project based on your requirements:

Step 1: Project Setup & Environment

1. **Prerequisites**: Install an IDE (like Eclipse Enterprise Edition or IntelliJ IDEA Ultimate), Apache Tomcat server (v9.0 or v10+), and MySQL Server.

2. **Project Structure**: Create a Dynamic Web Project in your IDE.

3. **Dependencies**: Add the MySQL JDBC Driver JAR file to your project's WEB-INF/lib folder (or configure it via Maven/Gradle if using build tools).

Step 2: Database Design (MySQL)

Create a database and a table to handle user data. Run the following SQL queries in your MySQL console:

SQL

```
CREATE DATABASE student_db;
USE student_db;

CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Step 3: Frontend Development (HTML & Bootstrap)

Create two main pages in your WebContent (or webapp) folder:

1. register.html **(Registration Page)**:
    - Use HTML5 and include **Bootstrap 5 CDN** in the <head> for styling.
    - Create a form with fields for Full Name, Email, and Password.
    - Add a submit button and link a custom JavaScript file (script.js).
2. login.html **(Login Page)**:
    - Create a clean login form with Email and Password inputs, styled with Bootstrap.

Step 4: Client-Side Validation (JavaScript & DOM Manipulation)

Write a JavaScript file (script.js) to validate form entries using DOM manipulation before they are sent to the server:

1. Fetch form input elements using DOM methods (e.g., document.getElementById).
2. Add an event listener to the form's submit event:
    - Check if fields are empty.

- Validate email format using Regular Expressions (Regex).

- Ensure the password meets length requirements.

- If validation fails, use DOM manipulation to dynamically display error messages under the respective fields and prevent form submission (event.preventDefault()).

Step 5: Backend Database Connectivity (JDBC Utility)

Create a Java class (DBConnection.java) to establish a connection to your MySQL database:

Java

```
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {
   public static Connection getConnection() {
      Connection con = null;
      try {
         Class.forName("com.mysql.cj.jdbc.Driver");
         con = DriverManager.getConnection("jdbc:mysql://localhost:3306/student_db", "root", "your_password");
      } catch (Exception e) {
         e.printStackTrace();
      }
      return con;
```

```
    }
}
```

Step 6: Backend Servlets Implementation

Create two Java Servlets to handle business logic and database operations:

1. RegisterServlet.java:
    - Map it using @WebServlet("/RegisterServlet").
    - Extract user inputs (name, email, password) using request.getParameter().
    - Use JDBC (PreparedStatement) to insert the data into the students table.
    - Redirect the user to login.html upon successful registration.
2. LoginServlet.java:
    - Map it using @WebServlet("/LoginServlet").
    - Verify the entered email and password against the database records.
    - **Session-Based Login**: If credentials match, create a session:

Java

```
HttpSession session = request.getSession();
session.setAttribute("userEmail", email);
response.sendRedirect("dashboard.jsp");
```

- If authentication fails, redirect back to login with an error message.

Step 7: Protected Dashboard & Session Management

1. Create a protected page (dashboard.jsp or welcome.jsp) that displays user-specific data.

2. Implement session checking at the top of the page to prevent unauthorized direct URL access:

Java

```
<%
  if(session.getAttribute("userEmail") == null) {
    response.sendRedirect("login.html");
  }
%>
```

3. Add a **Logout** mechanism via a Servlet or script that invalidates the session (session.invalidate()) and redirects back to login.html.